Student Management System

A Project Report

Submitted By:

MANGALA KALYANI

22351A6118

Department:

COMPUTER SCIENCE AND ENGINEERING(AI&ML)

College Name:

Dr.SAMUEL GEORGE INSTITUTE OF ENGINEERING AND TECHNOLOGY

Table of Contents

- Abstract3
- Chapter 1: Introduction4
- Chapter 2: System Requirements5
- Chapter 3: System Design6
- Chapter 4: Implementation (Source Code)7
- Chapter 5: Function Description9
- Chapter 6: Testing11
- Chapter 7: Output Screenshots12
- Chapter 8: Advantages and Disadvantages14
- Chapter 9: Future Enhancements15
- Chapter 10: Conclusion and References16

Abstract

The Student Management System is a console-based application developed using the Python programming language. In the modern educational landscape, managing student data efficiently is crucial for administrative success. Traditional paper-based methods are prone to errors, data loss, and inefficiency. This project aims to digitize the record-keeping process, providing a robust platform for adding, viewing, searching, deleting, and updating student information.

The system utilizes standard Python data structures, specifically lists and dictionaries, to store student records in memory during runtime. It features a user-friendly menu-driven interface that allows administrators to perform CRUD (Create, Read, Update, Delete) operations seamlessly. The simplicity of the code makes it an excellent educational tool for understanding the basics of database management logic without the complexity of an external database engine.

This report details the complete development lifecycle of the Student Management System, including the initial problem analysis, system design, implementation details, testing methodologies, and potential future improvements.

Chapter 1: Introduction

1.1 Problem Statement

Educational institutions handle a vast amount of student data, including personal details, academic performance, and enrollment information. Manual management of this data leads to several issues:

- High probability of human error during data entry.
- Difficulty in retrieving specific student records quickly.
- Redundancy and inconsistency in data storage.
- Risk of physical damage to paper records.

There is a need for a lightweight, digital solution to automate these tasks and ensure data integrity.

1.2 Objectives

The primary objectives of this project are:

- To design a Python-based application for managing student records efficiently.
- To implement a menu-driven interface that is easy to navigate for users.
- To provide core functionalities: Adding, Displaying, Searching, Deleting, and Updating records.
- To demonstrate the practical application of Python's list and dictionary data structures.

1.3 Scope

The scope of this project is limited to a console-based application running on a local machine. It stores data temporarily in the Random Access Memory (RAM) using Python lists. The system allows for:

- **Data Entry:** Inputting roll number, name, and branch.
- **Data Retrieval:** Viewing all students or searching by roll number.
- **Data Maintenance:** Updating specific fields or removing records.

The current scope does not include persistent storage (file handling or databases) or a graphical user interface (GUI), which are reserved for future enhancements.

Chapter 2: System Requirements

To successfully develop and run the Student Management System, specific hardware and software configurations are required.

2.1 Hardware Requirements

- **Processor:** Intel Core i3 or equivalent (Minimum), Intel Core i5 recommended.
- **RAM:** 2 GB Minimum (4 GB recommended for smooth operation of IDEs).
- **Hard Disk Space:** 100 MB free space for Python installation and project files.
- **Input Device:** Standard Keyboard.
- **Output Device:** Monitor with at least 1024x768 resolution.

2.2 Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Programming Language:** Python 3.x (Recommended: Python 3.8 or higher).
- **Code Editor / IDE:** VS Code, PyCharm, IDLE, or Sublime Text.
- **Interpreter:** Standard CPython interpreter.

Chapter 3: System Design

3.1 Data Flow

The system operates on a linear data flow model initiated by user input via the main menu.

1. **Start:** The program initializes an empty list `students = []`.
2. **Menu Display:** The `main_menu` function presents options to the user.
3. **Input Processing:** The user selects an operation (1-6).
4. **Function Execution:** Based on the choice, the corresponding function is called.
5. **Data Manipulation:** The global `students` list is accessed or modified.
6. **Loop:** The program returns to the main menu until the "Exit" option is selected.

3.2 Module Description

The system is modularized into distinct functions to ensure code readability and maintainability:

- **add_student():** Handles data input and appends a dictionary to the list.
- **display_students():** Iterates through the list to print all records.
- **search_student():** Performs a linear search based on Roll Number.
- **delete_student():** Finds and removes a dictionary from the list.
- **update_student():** Locates a record and modifies specific key-value pairs.
- **main_menu():** The controller function containing the infinite while loop.

3.3 Data Structure

The core data structure is a **List of Dictionaries**.

Structure format:

```
[
  {
    "roll": 101,
    "name": "Alice",
    "branch": "CSE"
  },
```

```
  {
    "roll": 102,
    "name": "Bob",
    "branch": "ECE"
  }
]
```

Chapter 4: Implementation

Below is the complete source code for the Student Management System.

```
# Global list to store student records
students=[]

def add_student():  # add Student record
   roll=int(input("Enter the roll number of the Student : "))
   name=input("Enter the name of the Student : ")
   branch=input("Enter the Branch of the Student : ")

   student={
      "roll":roll,
      "name":name,
      "branch":branch
   }
   students.append(student)
   print("Student added Sucessfully....!")

def display_students(): #display all students
   if len(students)==0:
      print("No Students Records found....!")
   else:
      print("_Student_Records_Are_")
      for i in students:
         print("Roll number is : ",i['roll'])
```

```
        print("Name is : ",i['name'])
        print("Branch : ",i['branch'])
        print("-----------------------------")
        print()

def search_student():
    search_roll=int(input("Enter the roll number to search : "))
    for i in students:
        if i['roll']==search_roll:
            print(i)
    print("No Record found with that roll no..")

def delete_student():
    del_search=int(input("Enter the roll no to delete student details : "))

    for i in students:
        if i['roll']==del_search:
            students.remove(i)
            print("Student record is deleted successfully...")
    print("No record with that roll number found to delete")

def update_student():
    update_roll=int(input("Enter the student roll number which should be updated."))
    for i in students:
        if i['roll']==update_roll:
            print("Enter the choice for updation")
            print("1. Name")
            print("2. Branch")
```

```
        print("3. Roll number")
        ch=int(input())
        if ch==1:
            new_name=input("Enter the new name : ")
            i['name']=new_name
            print("Name has updated successfully...")
        elif ch==2:
            new_branch=input("Enter the branch age to update : ")
            i['branch']=new_branch
            print("Branch is updated successfully...")
        elif ch==3:
            new_roll=int(input("Enter the new roll number "))
            i['roll']=new_roll
            print("Roll number updated successfullyy...")
        else:
            print("Invalid Choice entered Try again...")

def main_menu():
    while True:
        print("1. Add Student")
        print("2. Display Students")
        print("3. Search Student")
        print("4. Delete Student")
        print("5. Update Details")
        print("6. Exit")

        choice=int(input("Enter the number from 1-5 of your choice.."))
        if choice==1:
```

```
            add_student()
        elif choice==2:
            display_students()
        elif choice==4:
            delete_student()
        elif choice==3:
            search_student()
        elif choice==5:
            update_student()
        elif choice==6:
            print(".....Exit.....Thankyou.....")
            break
        else:
            print("Invalid choice ! Try again....")
main_menu()
```

Chapter 5: Function Description

1. add_student()

This function is responsible for creating new student records. It prompts the user to enter the Roll Number (integer), Name (string), and Branch (string). These details are packed into a dictionary object with keys "roll", "name", and "branch". This dictionary is then appended to the global `students` list. A success message "Student added Sucessfully....!" is displayed upon completion.

2. display_students()

This function lists all currently stored records. It first checks if the `students` list is empty using `len()`. If empty, it prints "No Students Records found....!". If records exist, it loops through each dictionary in the list and prints the Roll Number, Name, and Branch in a formatted manner, separated by dashed lines.

3. search_student()

This function facilitates the retrieval of a specific student's details. It asks the user for a "roll number to search". It then iterates through the `students` list comparing the input roll number with the 'roll' key of each dictionary. If a match is found, the entire student dictionary is printed. If the loop completes without finding a match, a "No Record found" message is displayed.

4. delete_student()

This function handles the removal of records. It takes a roll number as input. It iterates through the list to find a matching record. If found, the `students.remove(i)` method is called to delete that specific dictionary from the list, and a success message is printed. If the roll number is not found, an appropriate message is shown.

5. update_student()

This function allows modification of existing data. It first asks for the roll number to identify the student. If found, it presents a sub-menu asking what to update (Name, Branch, or Roll Number). Based on the user's secondary choice, it takes the new value and assigns it to the corresponding key in the dictionary.

6. main_menu()

This is the driver function of the program. It contains a `while True` loop that keeps the program running. It displays the 6 main options and captures the user's choice. Using `if-elif-else` conditional logic, it calls the appropriate function based on the input. Option 6 breaks the loop, terminating the program.

Chapter 6: Testing

The system was tested with various inputs to ensure validity and robustness.

Test Case ID	Function	Input Data	Expected Output	Actual Output	Status
TC_01	Add Student	Roll: 101, Name: John, Branch: CS	"Student added Successfully....!"	"Student added Successfully....!"	Pass
TC_02	Display	(List is not empty)	List of all students details	Displayed details of John	Pass
TC_03	Search	Roll: 101	{ 'roll': 101, 'name': 'John'... }	{ 'roll': 101, 'name': 'John'... }	Pass
TC_04	Search	Roll: 999 (Non-existent)	"No Record found..."	"No Record found..."	Pass
TC_05	Update	Roll: 101, Option: 1, New Name: Johnny	"Name has updated successfully..."	"Name has updated successfully..."	Pass
TC_06	Delete	Roll: 101	"Student record is deleted..."	"Student record is deleted..."	Pass
TC_07	Invalid Menu	Choice: 8	"Invalid choice ! Try again...."	"Invalid choice ! Try again...."	Pass

Chapter 7: Output Screenshots

The following are text-based simulations of the terminal output generated by the system.

7.1 Main Menu & Adding a Student

1. Add Student
2. Display Students
3. Search Student
4. Delete Student
5. Update Details
6. Exit

Enter the number from 1-5 of your choice..1
Enter the roll number of the Student : 101
Enter the name of the Student : Rahul
Enter the Branch of the Student : CSE
Student added Sucessfully....!

7.2 Displaying Students

Enter the number from 1-5 of your choice..2
_Student_Records_Are_
Roll number is : 101
Name is : Rahul
Branch : CSE

7.3 Searching for a Student

Enter the number from 1-5 of your choice..3
Enter the roll number to search : 101
{'roll': 101, 'name': 'Rahul', 'branch': 'CSE'}

7.4 Deleting a Student

Enter the number from 1-5 of your choice..4
Enter the roll no to delete student details : 101
Student record is deleted successfully...

Chapter 8: Advantages and Disadvantages

8.1 Advantages

- **Simplicity:** The code is straightforward, using only basic Python concepts, making it easy to understand and modify.
- **Efficiency:** Operations are performed in-memory, resulting in extremely fast execution times for small to medium datasets.
- **No External Dependencies:** The system uses only standard Python libraries; no database installation (like SQL) is required.
- **User-Friendly:** The menu-driven approach ensures that users can operate the system without knowing programming commands.
- **Cost-Effective:** Being open-source and lightweight, it requires minimal hardware resources.

8.2 Disadvantages

- **Data Volatility:** Since data is stored in a list (RAM), all records are lost when the program terminates. There is no file saving mechanism.
- **Scalability:** As the number of students increases, linear search algorithms used in `search` and `delete` functions will become slower (O(n) complexity).
- **Lack of Validation:** The system crashes if non-integer input is provided for the Roll Number. There is minimal error handling.
- **Console Interface:** The lack of a GUI makes it less visually appealing compared to modern applications.

Chapter 9: Future Enhancements

To overcome the current limitations and improve the system, the following enhancements are proposed:

1. Persistent Storage

Integrating File Handling (CSV or TXT files) or a database connection (SQLite/MySQL) to ensure that data is saved permanently and can be retrieved after restarting the application.

2. Graphical User Interface (GUI)

Developing a GUI using libraries like Tkinter or PyQt to replace the command-line interface. This would include buttons, input fields, and tables for a better user experience.

3. Input Validation

Implementing `try-except` blocks to handle ValueErrors (e.g., entering text when a number is expected) and ensuring logic checks (e.g., preventing duplicate roll numbers).

4. Advanced Search and Sorting

Adding functionality to search by Name or Branch and sorting the student list based on Roll Numbers or Names alphabetically.

5. Authentication

Adding a login system so that only authorized administrators can modify student records.

Chapter 10: Conclusion and References

10.1 Conclusion

The Student Management System project successfully demonstrates the capabilities of Python for handling basic data management tasks. By implementing core CRUD operations, the system addresses the fundamental need for organizing student information. While the current version operates in-memory, it serves as a solid foundation for more complex database-driven applications. The project highlights the importance of modular programming and data structuring in software development.

10.2 References

1. Python 3.10 Documentation. (n.d.). Retrieved from https://docs.python.org/3/
2. Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.
3. Downey, A. B. (2015). *Think Python: How to Think Like a Computer Scientist* (2nd ed.). O'Reilly Media.
4. W3Schools Python Tutorial. (n.d.). Retrieved from https://www.w3schools.com/python/